

American International University-Bangladesh (AIUB)

Faculty of Science and Technology

Department of Computer Science

Web Technologies

CSE 4101 / [Course Code]

PROJECT REPORT

Project Title:

Online Car Rent Management System

Semester: Spring 2025-26

Submission Date: 19.05.2026

Section: Q

Submitted To:

MD. AL-AMIN

Assistant Professor, Faculty

Department of Computer Science

American International University-Bangladesh

Group Information

Fill in all fields below. Each group member must sign before submission.

Group Number: G5

Project Title: Online Car Rent Management System

Student Information:

Full Name: MD. SADMAN ISLAM SHAMS

Student ID: 23-52568-2

Program: BSc in Computer Science and Engineering

Email: sadmanshams5000@gmail.com

Signature: _____

Declaration: We hereby declare that this project report is our original work. Any external resources or references used have been properly cited. We have not engaged in any form of academic dishonesty.

Marks Sheet

To be filled by the instructor only. Students must NOT write in this section.

Group No.		Section	
Member 1 ID			

CO3: Illustrate Multi-Tier Web Application for Targeted Society					
Feature Implementation (0-3)			Feature Implementation (0-3)		Completeness (0-3)
Basic Web Security	UI (HTML/CSS)	Feature Completeness	DB	Auth (Session/cookie)	MVC
CO4: Coordinate with Group Members to Develop Multi-Tier Application					
Technical Knowledge (0-3)			Collaborative Teamwork (0-3)		Communication/Promptness (0-3)
JS validation	PHP Validation	Ajax/JSON	Git Contribution		VIVA
Expected Outcome (0-2)		CO3 Total	CO4 Total		Project Total

Evaluated By: _____ **Date:** _____

Course Teacher Signature: _____

Table of Contents

1. Group Information	2
2. Marks Sheet	3
3. Table of Contents	4
4. Chapter 1: Project Overview	5
4.1 Project Description	
4.1 Technologies Used	
4.3 System Architecture / MVC Diagram	
4.4 Database Schema	
5. Chapter 2: Git Contribution & Task Allocation	7
5.1 GitHub Repository Link	
5.2 Git Contribution Screenshot	
6. Chapter 3: Features, Source Code & Screenshots	8
7. Chapter 5: Conclusion & Future Work	12

Chapter 1: Project Overview

1.1 Project Description

The Online Car Rent Management System is a web-based application where users can browse cars and place rental orders, while admins can manage the car inventory and monitor the system. The system is useful for a car rental business because it keeps car information, member records, order history, and blog content in one organized web application.

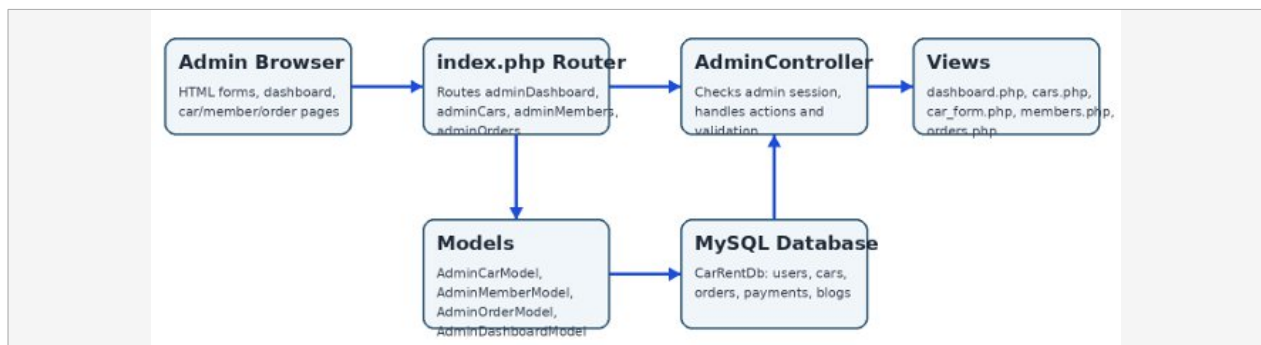
1.2 Technologies Used

List all technologies, frameworks, and tools used in this project.

Layer	Technology / Tool	Purpose
Frontend	HTML5, CSS3	UI structure and styling for admin pages
Client Script	JavaScript	AJAX member deletion and client-side interaction
Backend	PHP	Server-side request handling and validation
Database	MySQL	Store users, cars, orders, payments, and blogs
Auth	PHP Sessions / Cookies	Admin role-based access check
Version Control	Git / GitHub	Branching, commits, push, and pull request workflow
Server	XAMPP / Apache	Local development and testing

1.3 System Architecture / MVC Diagram

Insert your MVC architecture diagram or ER diagram here.



1.4 Database Schema

List your database tables and their key fields.

Table Name	Field Name	Data Type	Description
cars	id	INT	Primary Key
	name	VARCHAR(100)	Not Null
	model	VARCHAR(100)	Not Null
	type	enum	Not Null
	Ptice_Per_Day	decimal	Not Null
	Image_path	VARCHAR(255)	Null
	description	Text	Null

Chapter 2: Git Contribution & Task Allocation

2.1 GitHub Repository Link

Repository URL: <https://github.com/Rifat-Miah/online-car-rent-management-system.git>

2.2 Git Contribution Screenshot

Insert a screenshot of your GitHub contribution graph / commit history here.

The screenshot displays the GitHub repository interface for 'online-car-rent-management-system'. The top section shows the commit history with filters for Conversation (0), Commits (9), Checks (0), and Files changed (21). The commit history is organized by date, showing commits from May 16, 2026, to May 18, 2026. The commits are categorized by task, such as 'Task2: add admin dashboard with role-based access' and 'Task2: add admin car listing page'. The bottom section shows a pull request titled 'Task 2: Admin car management, member deletion, and rent order history #3' by 'sadmanshams369'. The pull request details the changes made, including the addition of 9 commits. The commit history is visualized as a vertical timeline of commits, each with a commit message and a commit hash. The pull request also includes a 'No conflicts with base branch' message, indicating that the merge can be performed automatically.

Conversation 0 Commits 9 Checks 0 Files changed 21 +1,777 -23

Commits on May 16, 2026

- Task2: add admin dashboard with role-based access bcd3468
- Task2: add admin car listing page 777e005

Commits on May 17, 2026

- Task2: add car creation with validation and image upload 47369ee
- Task2: add edit car functionality 43817cc

Commits on May 18, 2026

- Task2: add protected car deletion ba6ad6b
- Task2: add AJAX member deletion d1a92aa

Task 2: Admin car management, member deletion, and rent order history #3

sadmanshams369 wants to merge 9 commits into main from feature/task2-23-52568-2

sadmanshams369 added 9 commits 2 days ago

- Task2: add admin dashboard with role-based access bcd3468
- Task2: add admin car listing page 777e005
- Task2: add car creation with validation and image upload 47369ee
- Task2: add edit car functionality 43817cc
- Task2: add protected car deletion ba6ad6b
- Task2: add AJAX member deletion d1a92aa
- Task2: add rent order history page df5e37d
- Task2: add order history filters 9a2d837
- Task2: finalize admin module implementation 96468cd

No conflicts with base branch

Merging can be performed automatically.

Notifications

Unsubscribe

You're receiving notifications because you authored the thread.

1 participant

Lock conversation

Chapter 3: Features, Source Code & Screenshots

This chapter describes the features that I implemented for Task 2. I tried to keep my code organized in MVC style: controller logic in controllers/AdminController.php, database logic in model files, and UI in views/admin/.

Duplicate the feature section (Feature 1 block) as many times as needed — one block per feature.

Feature 1: Admin Dashboard and Access Control

Describe what this feature does and its purpose in the application.

Description

The admin dashboard gives a quick overview of the system. It displays total cars, total members, total orders, and total blog posts. The admin pages also use a role check so that only an admin session can access them.

Relevant Source Code

Paste the key source code snippet(s) below. Add the file path and language above each block.

File: controllers/AdminController.php

```
private function requireAdmin()
{
    $role = $_SESSION['user_role'] ?? $_SESSION['role'] ?? null;

    if ($role !== 'admin') {
        echo "Access denied. Please login as admin to view this page.";
        exit();
    }
}

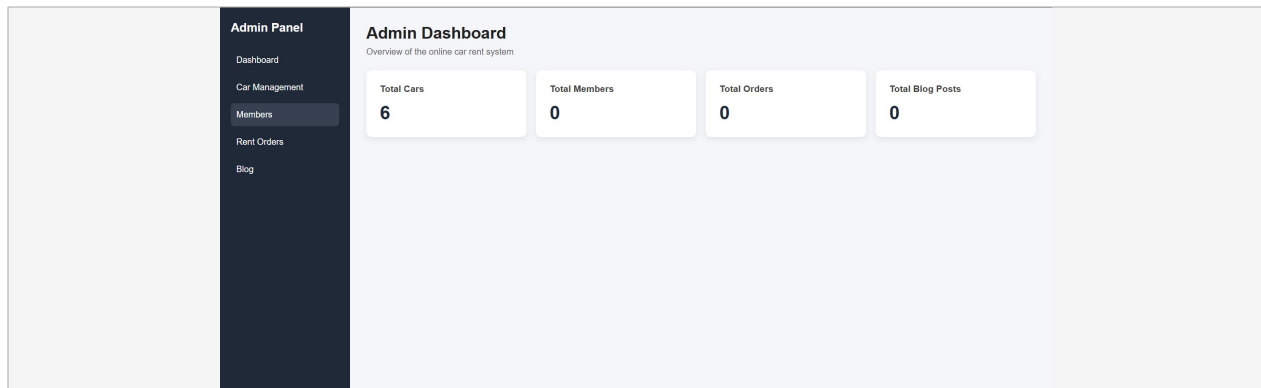
public function dashboard()
{
    $this->requireAdmin();
    $totalCars = $this->dashboardModel->getTotalCars();
    $totalMembers = $this->dashboardModel->getTotalMembers();
    $totalOrders = $this->dashboardModel->getTotalOrders();
    $totalBlogs = $this->dashboardModel->getTotalBlogs();
    require_once __DIR__ . '/../views/admin/dashboard.php';
}
```

File: models/AdminDashboardModel.php

```
public function getTotalCars()
{
    $sql = "SELECT COUNT(*) AS total FROM cars";
    $stmt = $this->conn->prepare($sql);
    $stmt->execute();
    $result = $stmt->get_result()->fetch_assoc();
    return $result['total'] ?? 0;
}
```

Screenshot(s)

Insert relevant screenshots of the UI for this feature below. Right-click > Insert Picture, or drag and drop.



Code Explanation

- 📄 requireAdmin() checks the session role before loading any admin page.
- 📄 The dashboard() function collects summary counts from the dashboard model.
- 📄 The model uses MySQLi prepared statements and returns safe count values.
- 📄 The dashboard view presents the count values in simple summary cards.

Feature 2: Car Management CRUD

Description

This feature allows the admin to manage the car inventory. I implemented car listing, add car, edit car, and protected delete. The car form validates required fields, validates positive price, validates allowed car type, validates image type and image size, and stores uploaded images in public/uploads/cars/.

Relevant Source Code

File: controllers/AdminController.php

```
private function validateCarInput($data)
{
    $errors = [];
    $allowedTypes = ['Private car', 'Microbus', 'PickUp', 'SUV', 'Van', 'Sedan'];
    $allowedStatuses = ['available', 'unavailable'];

    if (empty(trim($data['name'] ?? ''))) {
        $errors['name'] = 'Car name is required.';
    }
    if (empty(trim($data['model'] ?? ''))) {
        $errors['model'] = 'Car model is required.';
    }
    if (empty($data['type']) || !in_array($data['type'], $allowedTypes)) {
        $errors['type'] = 'Please select a valid car type.';
    }
    if (empty($data['price_per_day']) || !is_numeric($data['price_per_day']) || $data['price_per_day'] <= 0) {
        $errors['price_per_day'] = 'Price per day must be greater than 0.';
    }
    if (empty($data['availability_status']) || !in_array($data['availability_status'], $allowedStatuses)) {
        $errors['availability_status'] = 'Please select a valid availability status.';
    }
    return $errors;
}
```

File: models/AdminCarModel.php

```

public function createCar($data)
{
    $sql = "INSERT INTO cars
        (name, model, type, price_per_day, availability_status, image_path, description)
        VALUES (?, ?, ?, ?, ?, ?, ?)";
    $stmt = $this->conn->prepare($sql);

    $stmt->bind_param(
        "sssdsss",
        $data['name'],
        $data['model'],
        $data['type'],
        $data['price_per_day'],
        $data['availability_status'],
        $data['image_path'],
        $data['description']
    );
}

```

File: models/AdminCarModel.php

```

public function carHasActiveOrders($id)
{
    $sql = "SELECT COUNT(*) AS total FROM orders
        WHERE car_id = ? AND status IN ('pending', 'confirmed')";
    $stmt = $this->conn->prepare($sql);
    $id = (int) $id;
    $stmt->bind_param("i", $id);
    $stmt->execute();
    $result = $stmt->get_result()->fetch_assoc();
    return ($result['total'] ?? 0) > 0;
}

```

Code Explanation

- 📖 The validation function checks that the admin provides all required car details.
- 📖 Only the car types available in the shared database enum are accepted, including Pickup.
- 📖 The createCar() method inserts a car by using MySQLi prepared statements.
- 📖 The edit/update function reuses the same form and pre-fills the existing car data.
- 📖 The delete function checks active orders first. If a car has pending or confirmed orders, deletion is blocked to protect order data.
- 📖 Uploaded images are validated and stored in public/uploads/cars/.

Screenshot(s)

Insert relevant screenshots of the UI for this feature below. Right-click > Insert Picture, or drag and drop.

Admin Panel
Dashboard
Car Management
Members
Rent Orders

Car Management

View all car listings

Add New Car

ID	Image	Car Name	Model	Type	Price/Day	Status	Description	Created At	Action
1		Toyota Corolla	2023	Private car	2500.00	available	Family Car	2026-05-16 19:41:42	Edit Delete
2		Toyota Hiace	2022	Microbus	4500.00	available	Group Travel Car	2026-05-16 19:41:42	Edit Delete
3		Toyota Hilux	2023	PickUp	3500.00	available	Pickup Truck	2026-05-16 19:41:42	Edit Delete
4		Toyota Land Cruiser	2023	SUV	6000.00	available	Premium Car	2026-05-16 19:41:42	Edit Delete
5		Toyota Premio	2021	Sedan	2800.00	available	Premium Business Car	2026-05-16 19:41:42	Edit Delete
6		Toyota Noah	2022	Van	3800.00	available	Family Van	2026-05-16 19:41:42	Edit Delete

Feature 3: Member Management with AJAX Delete

Description

This feature lists all users whose role is member. The admin can delete a member using an AJAX request. The page does not need to reload after deletion, and the server returns a JSON response.

Relevant Source Code

File: models/AdminMemberModel.php

File: controllers/AdminController.php

Code Explanation

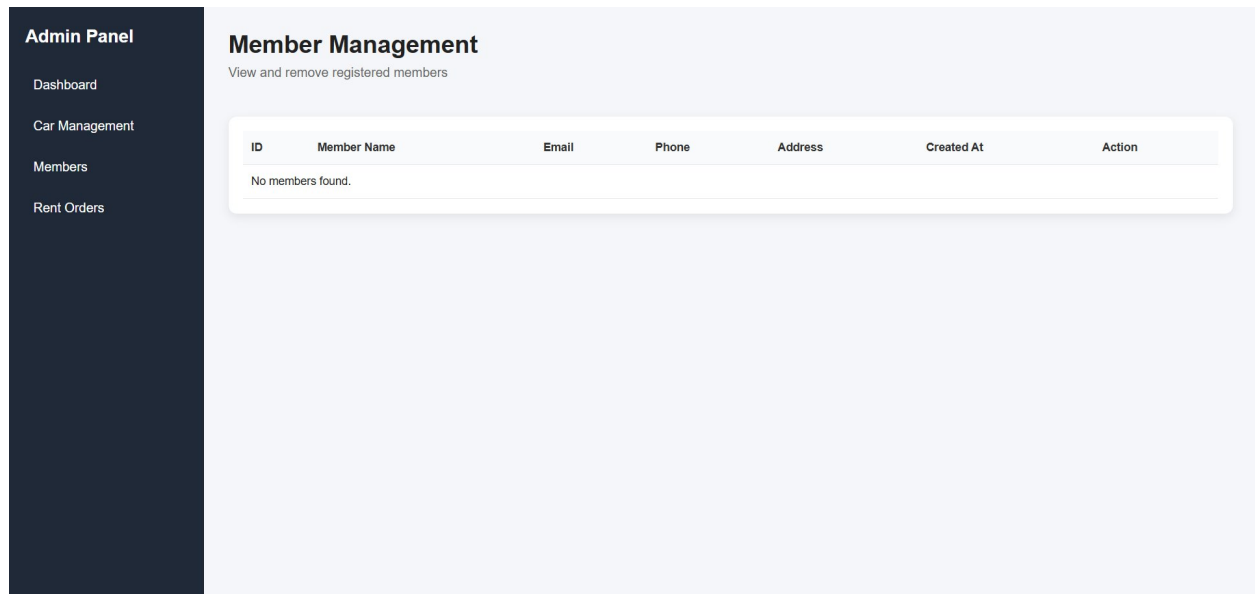
🎬 `getAllMembers()` only selects users with role = member, so admin users are not shown for deletion.

🎬 `deleteMember()` also checks role = member to avoid deleting an admin by mistake.

🎬 The controller returns JSON so that JavaScript can handle the result without a full page reload.

🎬 The JavaScript removes the member row from the table after a successful delete response.

🎬 This feature helped me satisfy the Ajax/JSON part of the grading criteria.



Chapter 5: Conclusion & Future Work

5.1 Summary

In this project, I completed the Task 2 admin module for the Online Car Rent Management System. My work includes the admin dashboard, car management CRUD, member deletion using AJAX, rent order history, and order history filters. I also aligned my code with the shared CarRentDb database and used MySQLi prepared statements for secure database operations. Through this task, I practiced MVC structure, server-side validation, AJAX handling, Git branching, regular commits, and pull request workflow.

5.2 Challenges Faced

- 🎬 Database schema mismatch: At first, the shared database schema had a different car type spelling. I checked the enum values and updated my code to match the final shared value `PickUp`.
- 🎬 Database connection change: My initial code used PDO, but later I converted my Task 2 models to MySQLi prepared statements to match the group direction.
- 🎬 Branch integration: The admin login route was not fully available in my Task 2 branch, so I kept a safe access denied message until Task 1 authentication is merged.
- 🎬 Testing delete functions: I had to be careful not to delete important data, so I tested member and car deletion using local test records.
- 🎬 Image paths: Seed car images and uploaded images used different path styles, so I adjusted the image display logic to support both.

5.3 Future Improvements

- Add a proper redirect from admin protected pages to the admin login page after the authentication branch is fully merged.

- Add pagination and search on the car list, member list, and order history pages for better usability when data grows.
- Add an admin order status update option so admin can update pending/confirmed/cancelled status directly from the order history page.

References

List all references in APA format. Include documentation, tutorials, libraries, and any other resources used.

[1] PHP Group. PHP Manual: mysqli. Retrieved from <https://www.php.net/manual/en/book.mysqli.php>

[2] MySQL. MySQL Documentation. Retrieved from <https://dev.mysql.com/doc/>

[3] GitHub Docs. About pull requests. Retrieved from <https://docs.github.com/en/pull-requests>